

配列

1. はじめに：コンピュータがデータを整理する理由

コンピュータの世界において、データはただ闇雲に詰め込まれているわけではありません。私たちが図書館で本を分類し、キッチンで調味料を整理するように、コンピュータもまた、情報を後から効率よく取り出し、活用するために「戦略的な整理」を行っています。この整理の仕組みを「データ構造」と呼びます。

データ構造とは、情報を単に蓄積するのではなく、後で使いやすくするための「棚」の作り方の設計図です。適切な棚（構造）がなければ、必要な情報を探すだけでコンピュータは膨大な時間を浪費し、プログラムの動作は重くなってしまいます。

数あるデータ構造の中で、プログラミングの第一歩として必ず登場するのが「配列」です。これは関連するデータを一行に並べて管理する、最もシンプルかつ強力な手法です。適切なデータ構造を選ぶことは、プログラムの「速さ」や「正確さ」に直結します。プロの視点から言えば、アルゴリズム（処理手順）が「料理のレシピ」なら、データ構造は「使いやすく整理された調理場」です。調理場が整っていないければ、どんなに優れたレシピもその真価を発揮できません。まずは、この配列の具体的なイメージを、身近な例え話から解き明かしていきましょう。

2. 配列のイメージ：連続した「ロッカー」のメタファー

配列の構造を直感的に理解するのに最適な例えが、CS50の講義でも紹介される「一行に並んだ番号付きの赤いロッカー」です。例えば、7つのデータを格納できる配列は、壁一面に「0番から6番まで」の扉が並んだロッカーのようなものと想像してください。

ここで最も重要なのは、データがメモリ（コンピュータの記憶領域）内で「隣り合って（連続して）」配置されているという点です。

- 一括管理：関連するデータがバラバラにならず、一つのグループとして扱える。
- 即時アクセス：データの塊が隣り合っているため、コンピュータは特定の場所を電光石火のスピードで特定できる。
- 順序の維持：データを入れた順番や並び順がそのまま保持される。

この「連続性」がなぜ重要なのか。もしロッカーが建物のあちこちに散らばっていたら、次のデータを確認するたびに廊下を歩き回るロスが発生します。しかし、配列はロッカーが隙間なく横に並んでいるため、手元をスライドさせるだけで次々とデータに触れることができます。この物理的な「近さ」が、コンピュータの処理効率における圧倒的なメリットを生んでいるのです。

3. 配列のルール：添字（インデックス）と0からのスタート

コンピュータの世界には、私たちが日常生活で使う「1番目、2番目……」という数え方とは異なる、厳格な標準ルールが存在します。ロッカーの扉に振られた番号を「添字（インデックス）」と呼びますが、プログラミングにおいてこの番号は必ず「0」から始まります。

これは単なる慣習ではなく、実務上の「標準言語」です。例えば、日本の「基本情報技術者試験」においても、2023年度（令和5年）の制度改正に伴い、試験で使われる疑似言語のルールが「1番始まり」から「0番始まり」へと変更されました。これは、C言語やPythonといった主要なプログラミング言語の実態に合わせるためです。

対象となる要素	私たちの日常的な感覚	コンピュータの視点（メモリのズレ）
最初のデータ	1番目	0 (先頭からのズレが0)
2番目のデータ	2番目	1 (先頭から1つ分隣)
n番目のデータ	n番目	$n - 1$

なぜ「0」なのか。それは、インデックスを「先頭からの距離（オフセット）」と捉えているからです。最初のロッカーは先頭そのものなので、距離は0。2番目のロッカーは先頭から1つ隣なので、距離は1となります。

このルールを理解することは、致命的なエラーを避けるために不可欠です。例えば、7個の要素を持つ配列（インデックス0～6）に対して、うっかり「7番」にアクセスしようとする、と、「セグメンテーションフォールト（Segmentation Fault）」というエラーが発生します。これはロッカーがない壁の部分無理やりこじ開けようとして、許可されていないメモリ領域を侵害したことを意味し、プログラムは強制終了してしまいます。

4. まとめ：配列から広がるデータ構造の世界

配列は、コンピュータがデータを扱うための最も基礎となる戦略的基盤です。

- 配列の本質は「連続したメモリ領域」であり、その物理的な隣接性が高速な処理を支えています。
- インデックスが「0」から始まるのは、先頭からのズレ（オフセット）を表す合理的なルールです。
- 配列という「構造」に、二分探索のような「アルゴリズム」を組み合わせることで、爆発的な効率化が実現します。

一方で、配列には限界もあります。通常、配列には同じ種類のデータ（数値なら数値のみ）しか入れられません。しかし、現実の世界では「名前」と「電話番号」をセットで扱いたい場面が多いでしょう。そこで、C言語の struct（構造体）のような、異なる型のデータをまとめて新しい型を作る仕組みが必要になります。CS50の phonebook.c の例にあるように、配列の中にこの「構造体」を入れることで、初めて実用的な住所録などが作れるようになるのです。